

使用数值数据

数值数据直接与数学相关，如：温度，重量，数量等。

特征向量

有效特征组成的**浮点值**数组，通常不会是由数据集的原始值组成，用于向模型输入特征信息。

特征工程（ML的重要组成部分）

数据集中的实际值的训练效果不一定好于经过更改的值的训练效果，所以需要将原始数据集中的值转化为可训练的值。

许多特征本质上是非数字值，此时就需要通过特征工程将其转化为数值。

常见技术包括：

- 归一化：将数值转化为标准范围。
- 分桶：将数值转化为范围分桶。

初始步骤

在创造特征向量前，需要**通过图形或图表直观观察数据、获取有关数据的统计信息**。

数据可视化

图表可以帮助我们发现数据中隐藏的异常或模式，这种可视化的方式有助于我们检验自己的假设。

可以使用 pandas 进行可视化。

数据的统计评估

通过数学方法评估潜在特征和标签，收集一些基本统计信息：平均值、中位数、标准差、四分位数分界点的值（第0,25,50,75和100分位）

查找离群值

离群值与特征或标签中的大部分其他值相差甚远，往往会导致模型训练出现错误。

如果第0百分位数与第25百分位数间的差异和第75百分位数与第100百分位数间的差异显著不同，则可能存在离群值。

当然，**平衡**的数据中也可能存在离群值。汇总统计数据有时会掩盖数据集中较小部分存在的问题。

离群值可能是错误值，也可能是合法的数据值，对于前者直接删除即可，对于后者则有两种处理方式：

- 若模型需要利用离群值，就保留。但极端离群值仍会影响训练。
- 若模型不需要，就删除，或运用其他技术（如裁剪）。

归一化

目标：将特征转化为相似的比例，从而让两个范围差异大的特征拥有相似的范围。

优势：有助于模型在训练时快速收敛；有助于在特征值非常高时避免“NaN”；有助于做出更精准的预测；有助于模型对每个特征做出适合的权重判断。

特征和标签都可以进行归一化，如果标签进行了归一化，那么预测结果也进行了归一化，在实际使用时需要进行反归一化。

常见方法：线性放缩、Z 分数放缩、对数放缩。

线性放缩（数据呈扁平状）

将浮点值转化为 $0 \sim 1$ 或 $-1 \sim +1$ 。

公式如： $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$ ，此时范围就被放缩为 $0 \sim 1$ 了。

当满足以下条件时，适合使用该方法：

- 数据的上下限受时间的影响不大。
- 数据中不存在或存在很少离群值，且离群值不极端。
- 数据在范围内大致均匀分布。

生活中的大部分特征不适合线性放缩。

Z 分数放缩（数据呈钟形）

Z 得分：某个值与平均值相差的标准差数（有正负）。

该放缩意味着将 Z 得分存储在特征向量中。

公式如： $x' = \frac{x - \mu}{\sigma}$ （ μ 是平均值， σ 是标准差）

若数据遵循**正态分布**或类似分布，则该方法适用。

若大部分值符合正态分布但包含少数极端离群值，可以考虑使用 Z 分数放缩和其他形式的归一化（如剪裁）相结合。

对数放缩（数据呈重尾形）

计算原始值的对数，理论上底数任意，但通常会计算自然对数。

公式如： $x' = \ln(x)$ 。

当数据符合幂律分布时，该方法适用，可以改变数值分布，有助于训练出更好的预测模型。

若一个随机变量 x 出现的概率 $P(x)$ 与其本身大小的某次幂成反比，即满足公式 $P(x) = Cx^{-\alpha}$ （其中 C 是常数， α 是幂指数，通常在现实中 $2 < \alpha < 3$ ），我们就说 x **服从幂律分布**。

剪裁（数据包含极端离群值）

最大限度减少极端离群值影响的技术，通常会将离群值限制为特定的最大值。

可防止模型过度关注不重要的数据。

分桶

将不同的数据分到不同的箱或桶中，将数值数据转化为分类数据，从而减少不同取值的数量。桶中数据足够多时，才能让模型学习桶和标签间的联系。

当**特征与标签间总体的线性关系微弱甚至不存在**或**特征值聚类**时，可以考虑该技术。

分位数分桶

创建分桶边界，使每个桶中的样本数量几乎相等，可以隐藏离群值。

等间隔分桶适用于大部分场景，但当数据倾斜时，可以考虑分位数分桶。

擦除

由于存在一个或多个问题（如有值被省略、示例重复、特征值超出范围、标签有误等），需要编写脚本或程序来检测问题。

良好数值特征的特性

1. 名称明确。
2. 在训练前已检查或测试（排除不良数据）。
3. 明智（运用新的数组让模型理解某些值是缺失值）。

多项式转换

当一个变量与另一个变量的某次幂相关时，可以考虑从现有的某个数值特征中创建合成特征，此过程为多项式转换。

在转换后，按照线性回归问题的处理方式处理，依旧可以实现数据点的分割，从而对两个特征间的非线性关系建模。

处理分类数据

数值可以是数值数据，也可以是分类数据。

编码是为了将分类数据或其他数据转化为浮点值数据，从而让模型训练。

词汇和独热编码

维度相当于特征向量中的元素数量。

词汇

当分类特征的可能类别数量较少时，可以将其编码成**词汇**，而用词汇表编码时，模型会把每一个可能的分类值视为一个**单独的特征**，赋予不同权重。

在将字符串转化为唯一索引后，需要进一步处理，防止模型将其视作单纯的浮点数。

独热编码

构建词汇表后，就需要将每一个索引转化为对应的**独热编码**，以传递给特征向量，让模型为每个元素制定合适的权重。

在经过独热编码后，特征维度有可能增加。

独热编码满足以下性质：

- 每个类别都由一个包含 N （类别数量）个元素的向量（数组）表示。
- 一个元素的值为 1.0，其余元素的值为 0.0。

稀疏表示法

如果某个特征的值主要为零或空，则成该特征为**稀疏特征**，许多分类特征都是稀疏特征。

稀疏表示法指表示在稀疏向量中存储 1.0 的位置。

稀疏表示法所用内存少于多元素独热向量，但是模型必须用独热向量来训练。

多热编码的稀疏表示法会存储**所有**非零元素的位置。

分类数据中的离群

分类数据中也包含离群值，往往将离群值归入一个名为**词汇表外 (OOV)** 的类别，即归入一个离群值专属的桶中，让模型给这个桶设定合适的权重。

对高维分类特征进行编码

类别数量较多时，不适合使用独热编码，通常会使用嵌入。

嵌入可以大幅度减少维度，有两大优势：

- 训练速度通常更快。
- 模型可以更快地给出预测结果，即延迟时间更短。

也可以使用哈希。

常见问题

数值数据通常由科学仪器或自动化测量记录；分类数据通常按照人类或机器学习模型进行分类，故决定类别和标签的人员以及他们做出决定的方式会影响数据的可靠性和实用性。

人工评分员

由人工手动标记的数据通常称为**标准答案**，其质量相对高，因此在训练模型时，会比机器标记的数据受欢迎。

但这并不意味着人工手动标记的数据的质量一定高，认为错误、恶意、偏见可能会干涉数据收集、清理和处理过程，所以在训练前需要检查。

人工评价者之间的评价结果差异被称为**评价者间一致性**。

机器评分者

机器标记的数据（类别由一个或多个分类模型自动确定）通常称为**银标签**。

此类数据在质量上良莠不齐，不仅要检查准确性和偏见，还要检查是否违背常识、现实和意图。

在数据检查前，需要先了解数据中机器标签和注释的质量和偏差。

高维度

分类数据往往会产生高维特征向量（即包含大量元素），而高维度会增加训练费用和训练难度，因此在训练前需要减少维度。

对于自然语言数据，降维的主要方式是将特征向量转化为嵌入向量。

特征组合

通过对数据集中两个或更多分类特征或分桶特征进行组合（如取笛卡尔积，将不同组的数值相乘）创建特征组合，从而让线性模型处理非线性问题。

多项式转换组合数字数值，特征交叉组合分类数据。

可以使用神经网络在训练期间自行发现并应用有用的特征组合。

对两个稀疏特征进行交叉会产生更加稀疏的新特征。

数据集、泛化和过拟合

数据集

数据集是一组示例，通常会以表格的形式存储数据，也可能是从其他格式派生而来。

表格是一种直观的机器学习模型的输入格式。

数据类型

数值数据、分类数据、人类语言、多媒体内容、其他机器学习系统的输出、嵌入向量。

数据数量

一般情况下，模型的训练样本应该至少比可训练参数多一个数量级，而优质模型通常会使用更多样本来训练。

在特征较少的大型数据集上训练的模型通常比在特征较多的较小数据集上训练的模型效果好。

不同问题需要的数据集大小不同。

对于已经使用大量数据训练过的模型，则可以使用小型数据集来获得良好的效果。

数据的质量和可靠性

高质量的数据有利于模型实现其目标，低质量的数据会妨碍模型实现其目标。

优质的数据通常也比较可靠，**可靠性**是指对数据的信任度。

衡量可靠性时，必须确定以下内容：

- 标签错误有多常见？
- 特征是否噪声过大，或者说特征中的值是否有错误？当然，我们无法从数据集中删除所有的噪声，所以出现一些噪声是正常的。
- 数据是否根据问题进行了筛选？

导致数据不可靠的常见原因如下：

- 遗漏值。
- 重复示例。
- 不良特征值。
- 不良标签，如标记错误。
- 数据有部分损坏。

建议使用自动化功能标记不可靠的数据。

任何足够大或多样化的数据集几乎肯定都包含离群值，确定如何处理离群值是机器学习的重要组成部分。

完整示例和不完整示例

当示例不完整时，应删除未完成的示例，或者合理推测缺失值从而让示例完整。

完整示例足够多，或特征对模型帮助不大时，应直接删除不完整示例，但不要删除重要的示例。

如果无法确定示例是否重要，可以构建两个数据集，一个通过删除不完整示例构建，一个通过插值构建，然后确定哪个数据集训练出来的模型效果更好。

加入插值是生成合理数据的过程，并不会生成随机数据或欺骗性数据。良好的插值可以改进模型，不当的插值可能损害模型。常见做法是使用平均值或中位数作为插值，而当使用 Z 得分来表示数值特征时，插入值通常为 0，因为 0 通常是平均 Z 得分。

处理插值时，应告诉模型哪些是实际值，哪些是插值，故可以添加一个布尔值列。

排序的数据集可以简化插值，但是不能用已经排好序的数据集训练模型，因此在插补后需要将数据集打乱。

标签

直接标签与代理标签

直接标签：与模型尝试做出的预测完全相同的标签，即模型尝试做出的预测恰好以列的形式存在于数据集中。

代理标签：与模型尝试做出的预测相似但不完全相同的标签。

直接标签通常比代理标签更好，不过直接标签通常无法直接使用。

使用代理标签是一种折中的方法，其与预测的关联越强，训练出来的模型效果越好。

当直接标签难以直接用于训练时，就需要使用代理标签。

人工生成的数据

有些数据是人工生成的，一个或多个人检查某些信息并提供一个值（通常是标签）。

还有些数据是自动生成的，由软件（可能是另一个机器学习模型）确定的值。

优点：

- 人工评估者可以执行各种各样的任务，而有些任务机器学习模型难以完成。
- 数据集的所有者可以制定清晰一致的标准。

缺点：

- 通常需要向人工评估者付费，因此人工生成的数据可能很昂贵。
- 人有可能出错，所以需要多个人评估员来评估同一数据。

在考虑人工生成的数据时，应该考虑以下内容：

- 评分员应该具备哪些技能？
- 需要多少个带标签的示例？多久之后需要数据？
- 预算是多少？

务必仔细检查人工评估员的评估结果。

模型可以使用混合数据训练。

类别不平衡的数据集

类别平衡的数据集与类别不平衡的数据集

假设一个数据集中包含一个分类标签，其值为正类别和负类别，那么在**类别平衡**的数据集中两种类别数量大致相等，而在**类别不平衡**的数据集中一个标签的数量比另一个标签的多很多。

在现实生活中，类别不平衡的数据集更为常见。

在类别不平衡的数据集中，较常见的类别被称为**多数类**，较不常见的类别被称为**少数类**。

训练类别严重不平衡的数据集的难度

训练旨在创建一个可以正确区分正类别和负类别的模型，因此，训练的批次中需要有足够数量的正面类和负面类。当数据集中的类别严重不平衡时，就会导致没有足够的少数类示例来训练模型，批次中的少数类示例过少，模型无法得到正常的训练。

类别严重不平衡时，准确度往往会被误导，此时更应关注精确率、召回率和 F_1 分数

用类别不平衡的数据集训练

在训练时，模型应学会两件事：

- 每个类别的外观，即哪些特征值对应哪些类别。
- 每个类别的常见程度，即类别的相对分布情况。

标准模型通常会混淆这两个目标，故我们需要两种技术（**对多数类进行下采样和上调权重**）来分离这两种目标，使模型能实现这些目标。

对多数类进行下采样和上调权重在某种程度上是违反直觉的。

应该尝试不同的下采样和上调权重系数，就像尝试其他超参数一样，从而训练出效果做好的模型。

对多数类进行下采样

下采样是指使用占比异常低的多类样本进行训练，通过从训练中省略许多多数类示例，人为地强制类别不平衡的数据集变得更加平衡。

下采样可以大幅度提高每个批次中包含少数类示例的概率，从而能够正确高效地训练模型。

增加下采样后类别的权重

下采样会向模型展示比现实世界更平衡的人工世界，从而引入**预测偏差**。为了修正这种偏差，就必须上调多数类的权重。

上调权重是指与少数类示例的损失相比，更严格地处理多数类示例的损失。

优势

- 更好的模型：生成的模型“知道”**特征与标签之间的关联和类别的真实分布**。
- 更快收敛。

拆分原始数据集

所有优秀的软件工程项目都会投入大量精力来测试其应用，因此我们应测试 ML 模型，以确定其预测的正确性。

训练集、验证集和测试集

应使用与训练模型所用示例不同的示例来测试模型。

与对同一组示例进行测试相比，对不同示例进行测试更能证明模型的适用性，而这些示例可以通过**拆分原始数据集**来获得。

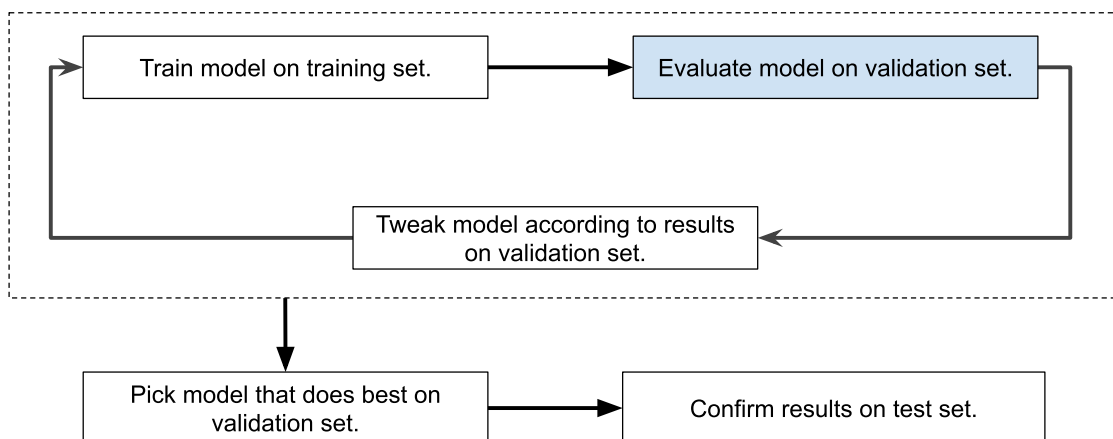
比如，可以把原始数据集拆分为模型进行训练用的**训练集**和用于评估训练好的模型的**测试集**。

但，一般会将原始数据集拆分为三个部分：**训练集**、**验证集**（在模型训练期间对其进行初始测试）和**测试集**。

使用验证集评估训练集中的结果，在反复使用验证集后，若模型的预测结果良好，再使用测试集仔细检查模型。

一般会有如下 workflow：

“调整模型”是指调整模型的任何方面。



拆分数据集后，只使用训练集计算归一化所需的数据，再使用相同的参数转换训练集、验证集、测试集和真实数据。

测试集和验证集在被反复使用后会“磨损”，所以最好收集更多的数据来“刷新”测试集和验证集。

测试集的其他问题

重复示例可能会影响模型的评估，所以在拆分后，需要删除验证集和测试集中与训练集中重复的所有示例。

对模型进行公平测评的唯一方法是使用**新示例**，而不是重复示例。

一组好的测试集应该满足以下所有条件：

- 足够大，可以得出具有统计显著性的测试结果。
- 能代表整个数据集，即挑选的测试集的特征应与训练集的特征相同。
- 代表模型在使用时会遇到的真实数据。
- 训练集中没有重复的示例。

转换数据

机器学习的一个重要部分就是**将非浮点特征转化为浮点值表示法**。

除了非浮点特征，浮点特征也需要进行转换，将其转换为受限范围以改进模型训练，此过程为**标准化**。

若数据集包含的样例过多，应当选择一组子集进行训练，而这组子集应是**与模型预测关联最强的一组**。

优质数据集会省略包含个人身份信息的示例，这有助于保护隐私，但可能会影响模型。

泛化

在机器学习模型中，可能会出现模型在训练用的特定组上表现良好，在真实世界中却表现糟糕，则这种模型没有**泛化**能力，过度拟合了训练数据。

过拟合

过拟合是指创建的模型与训练数据过于匹配，导致模型无法根据新数据做出正确的预判。

过拟合是机器学习中的常见问题，并非学术假设。

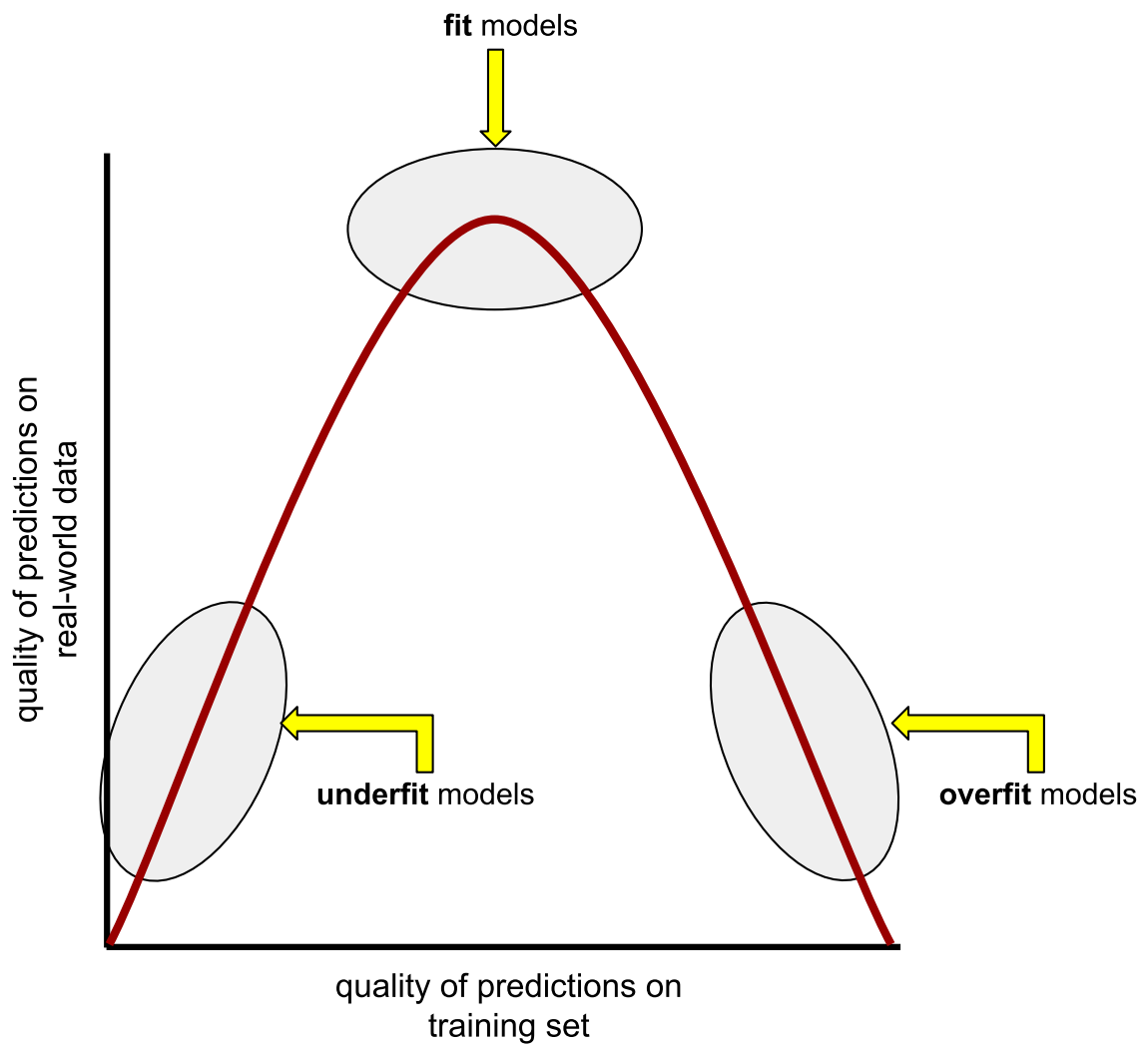
拟合、过拟合和欠拟合

模型必须对新数据做出良好的预测，即能“拟合”新数据。

欠拟合模型无法对训练数据做出准确的预判。

泛化与过拟合相反，泛化能力强的模型可以对新数据做出良好的预测。

拟合、过拟合和欠拟合关系如下图。



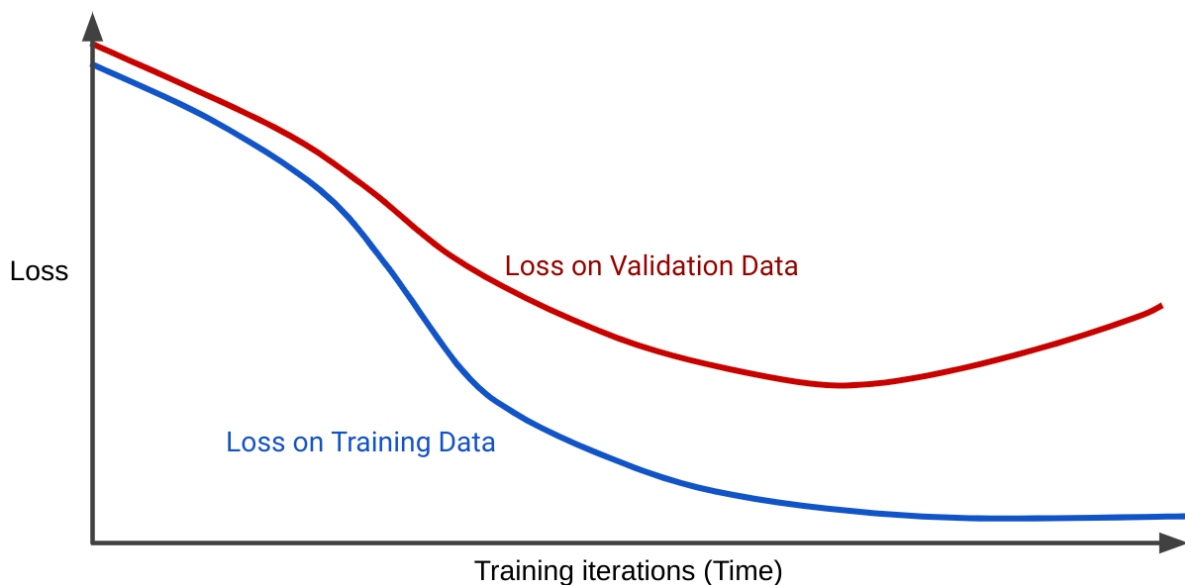
检测过拟合

以下曲线可以帮助我们检测过拟合：

- 损失曲线。
- 泛化曲线。

损失曲线会将模型的损失和训练迭代次数绘制在图表中。

泛化曲线显示两个或更多损失曲线。如下图泛化曲线显示了两个损失曲线。



此图中，最后训练值的损失下降或保持稳定但验证集的损失增加，表明模型过拟合。适合度高的模型的泛化曲线应当呈现两个性质相似的损失曲线。

过拟合的原因

一般来说，导致过拟合的原因如下：

- 训练集不能充分代表真实数据。
- 模型过于复杂。

泛化条件

训练模型，使其能够很好地泛化到新数据中，需要数据集满足以下条件：

- 示例必须独立且等概率分布。
- 数据集是平稳的，即不随时间的推移发生显著变化。
- 数据集分区具有相同的分布，即在统计上，训练集中的示例与验证集、测试集和真实数据中的示例相似。

模型复杂性

复杂模型的效果不一定优于简单模型。

复杂模型在训练集上的表现通常优于简单模型，而简单模型在测试集上的表现通常优于复杂模型。

正则化

机器学习模型必须满足两个互相冲突的目标：

- 能很好地拟合数据。
- 尽可能简单地拟合数据。

为了让程序简单，一种做法是惩罚复杂的模型，即强制模型简单化，这种做法是一种**正则化**。

损失和复杂性

训练的唯一目的是尽量减少损失，即 $\text{minimize}(\text{loss})$ 。

仅专注于最大限度减少损失的模型往往会过拟合，所以更好的算法应当最大限度减少损失和复杂度的某种组合，如： $\text{minimize}(\text{loss} + \text{complexity})$ 。

然而，损失和复杂度通常成反比，所以需要找到一个点，使模型对训练数据和真实数据都能做出良好的预测。

L_2 正则化

L_2 正则化 是一种常用的正则化指标，公式为： $L_2 \text{ regularization} = w_1^2 + w_2^2 + \dots + w_n^2$ 。

接近于零的权重对 L_2 正则化的影响不大，而较大的权重可能会产生巨大的影响。

L_2 正则化会使权重接近于 0，但权重不会完全为 0。

正则化率 (λ)

模型开发者可以通过将复杂度乘以一个称为**正则化率**的标量，来调整复杂度对模型训练的总体影响，通常用 λ 来表示正则化率。

所以，模型开发者想要实现的是： $\text{minimize}(\text{loss} + \lambda \text{ complexity})$ 。

较高的正则化率会产生以下影响：

- 增强正则化的影响，从而降低过拟合的可能性。
- 往往会生成近乎**正态分布、平均权值为 0** 的模型权重直方图。

较低的正则化率会产生以下影响：

- 降低正则化的影响，从而增加过拟合的可能性。
- 往往会生成几乎平坦分布的模型权重直方图。

选择正则化率

理想的正则化率会让模型能够很好地泛化到以前从未见过的数据，而该理想值取决于数据，所以必须进行一些调整。

早停法（基于复杂度的正则化的替代方法）

早停法是一种正则化方法，不涉及复杂度的计算，只是在模型完全收敛前结束训练。

早停法通常会增加训练损失，但可以减少测试损失，是一种快速但很少是最佳的正则化形式。

在学习速率和正则化率之间找到平衡点

学习速率和正则化率往往会使权重朝着相反方向移动，而我们的目标就是找到二者的平衡点，不停调整二者的大小。

解读损失曲线

损失曲线通常很难解读。

在调试训练问题时，通常最好降低学习速率。